

AI-Assisted Research on OpenRAM-Based 4KB SRAM Design Using SKY130 PDK

Bhavani Guni | VSD Internship | Week-01

Abstract— This work presents a Week-1 investigation of a 1024×32 SRAM (32 Kbits / 4 KB) using the OpenRAM memory compiler and SKY130 Process Design Kit (PDK). The study focuses on SRAM architecture, 6T SRAM cell operation, OpenRAM compiler flow, SKY130 integration, generated design views, verification methodology, and debugging strategies. An AI-assisted workflow was used to analyze the reference repository, understand the SRAM generation process, and identify potential implementation challenges. The investigation provides a foundation for future SRAM generation, verification, and optimization tasks.

Keywords— SRAM, OpenRAM, SKY130, 6T SRAM Cell, Memory Compiler, VLSI Design.

I. INTRODUCTION

Static Random Access Memory (SRAM) is widely used in processors, cache memories, and embedded systems due to its high-speed operation and low access latency. This investigation focuses on a 1024×32 SRAM (32 Kbits / 4 KB) designed using the OpenRAM memory compiler and SKY130 Process Design Kit (PDK) operating at 1.8 V. The objective of Week-1 is to understand the complete SRAM generation flow, including architecture, compiler configuration, technology integration, verification methodology, and AI-assisted debugging strategies before implementation.

II. SRAM ARCHITECTURE AND CELL OPERATION

The SRAM architecture consists of a row decoder, column decoder, bitcell array, precharge circuits, sense amplifiers, write drivers, and control logic. The memory array contains 1024 rows and 32 columns of 6T SRAM cells. Each 6T SRAM cell uses two cross-coupled CMOS inverters and two access transistors to store one bit of data. During read operation, precharged bitlines develop a small voltage difference that is amplified by the sense amplifier. During write operation, the write driver forces complementary values on BL and BLB to update the stored data. Figure 1 illustrates the SRAM array organization and the 6T SRAM cell structure.

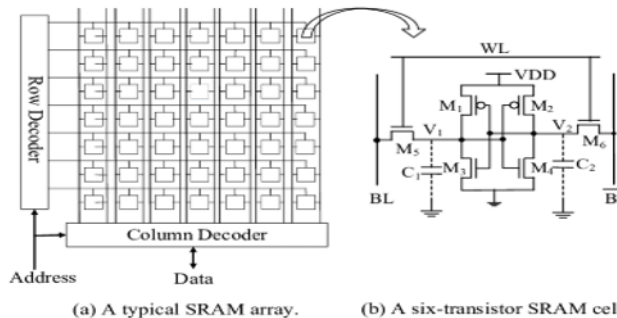


Fig. 1. Typical SRAM array architecture and six-transistor (6T) SRAM cell.

III. OPENRAM AND SKY130 INTEGRATION ANALYSIS

OpenRAM is an open-source SRAM compiler that automatically generates memory macros from user-defined specifications. The SKY130 PDK provides transistor models, design rules, layout layers, SPICE models, and verification decks required for SRAM generation. Technology integration is achieved through configuration files and environment variables such as PDK_ROOT. OpenRAM uses SKY130 technology information to generate technology-aware SRAM layouts and timing models suitable for fabrication and verification.

IV. COMPILER FLOW AND GENERATED OUTPUTS

The SRAM generation flow begins with a configuration file containing parameters such as word_size=32, num_words=1024, and tech_name=sky130. OpenRAM then creates the bitcell array, peripheral circuits, memory hierarchy, and physical layout. The compiler generates multiple design views including GDS for layout, SPICE for simulation, LEF for physical abstraction, LIB for timing characterization, and Verilog models for functional verification. These outputs support subsequent stages of design implementation and analysis.

V. VERIFICATION AND DEBUGGING STRATEGY

Verification is performed using both pre-layout and post-layout methodologies. Pre-layout simulations validate read, write, and hold operations without parasitic effects, while post-layout simulations evaluate realistic performance using extracted resistance and capacitance information. Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification ensure manufacturability and design correctness. Common issues include DRC violations, LVS mismatches, missing dependencies, and simulation convergence errors.

VI. AI-ASSISTED WORKFLOW AND KEY OBSERVATIONS

AI tools such as ChatGPT can accelerate SRAM research by decomposing the design flow into smaller learning tasks, explaining OpenRAM scripts, generating setup instructions, and suggesting debugging strategies. The adopted workflow involved repository analysis, SRAM architecture study, OpenRAM flow understanding, SKY130 integration analysis, verification planning, and report preparation. AI assistance improved learning efficiency while all generated information was independently verified.

VII. CONCLUSION

This study established a fundamental understanding of SRAM architecture, OpenRAM flow, SKY130 integration, verification methodology, and AI-assisted debugging approaches for future SRAM design tasks.